

5.7 Full-Netlist Routing

5.7

In order to successfully route multiple nets, global routers must properly match nets with routing resources, without oversubscribing resources in any part of the chip. All signal nets are either routed *simultaneously*, e.g., using (integer) linear programming (Sec. 5.7.1), or *sequentially*, e.g., one net at a time (Sec. 5.6). When certain nets cause resource contention or overflow for routing edges, sequential routing requires multiple iterations. These iterations are performed by *ripping up* the nets that cause violations (Sec. 5.7.2) and *rerouting* them with fewer violations. The iterations continue until all nets are routed without violating capacities of routing-grid edges or until a timeout is exceeded.

► 5.7.1 Routing by Integer Linear Programming

A *linear program* (LP) consists of a set of *constraints* and an optional *objective function*. This function is maximized or minimized subject to these constraints. Both the constraints and the objective function must be linear. In particular, the constraints form a system of linear equations and inequalities. An *integer linear program* (ILP) is a linear program where every variable can only assume integer values. ILPs where all variables are binary are called *0-1 ILPs*. (Integer) Linear programs can be solved using a variety of available software tools such as GLPK [5.7], CPLEX [5.13], and MOSEK [5.17]. There are several ways to formulate the global routing problem as an ILP, one of which is presented below.

The ILP takes three inputs – (1) an $W \times H$ routing grid G , (2) routing edge capacities, and (3) the netlist *Netlist*. For exploitation purposes, a horizontal edge is considered to run left to right – $G(i,j) \sim G(i+1,j)$ – and a vertical edge is considered to run bottom to top – $G(i,j) \sim G(i,j+1)$.

The ILP uses two sets of variables. The first set contains k Boolean variables $x_{net_1}, x_{net_2}, \dots, x_{net_k}$, each of which serves as an indicator for one of k specific paths or route options, for each net $net \in Netlist$. If $x_{net_k} = 1$, (respectively, $= 0$), then the route option net_k is used (respectively, not used). The second set contains k real variables $w_{net_1}, w_{net_2}, \dots, w_{net_k}$, each of which represents a net weight for a specific route option for $net \in Netlist$. This net weight reflects the desirability of each route option for net (a larger w_{net_m} means that the route option net_m is more desirable – e.g., has fewer bends). With $|Netlist|$ nets, and k available routes for each net $net \in Netlist$, the total number of variables in each set is $k \cdot |Netlist|$.

Next, the ILP formulation relies on two types of constraints. First, each net must select a single route (mutual exclusion). Second, to prevent overflows, the number of routes assigned to each edge (total usage) cannot exceed its capacity. The ILP maximizes the total number of nets routed, but may leave some nets unrouted. That

is, if a selected route causes overflow in the existing solution, then the route will not be chosen. If all routes for a particular net cause overflow, then no routes will be chosen and thus the net will not be routed.

Integer Linear Programming (ILP) Global Routing Formulation

Inputs:

W, H	:	width W and height H of routing grid G
$G(i, j)$	:	grid cell at location (i, j) in routing grid G
$\sigma(G(i, j) \sim G(i + 1, j))$	:	capacity of horizontal edge $G(i, j) \sim G(i + 1, j)$
$\sigma(G(i, j) \sim G(i, j + 1))$	:	capacity of vertical edge $G(i, j) \sim G(i, j + 1)$
$Netlist$	:	netlist

Variables:

$x_{net_1}, \dots, x_{net_k}$	:	k Boolean path variables for each net $net \in Netlist$
$w_{net_1}, \dots, w_{net_k}$	:	k net weights, one for each path of net $net \in Netlist$

Maximize:

$$\sum_{net \in Netlist} w_{net_1} \cdot x_{net_1} + \dots + w_{net_k} \cdot x_{net_k}$$

Subject to:

Variable Ranges:

$$x_{net_1}, \dots, x_{net_k} \in [0, 1] \quad \forall net \in Netlist$$

Net Constraints:

$$x_{net_1} + \dots + x_{net_k} \leq 1 \quad \forall net \in Netlist$$

Capacity Constraints:

$$\sum_{net \in Netlist} x_{net_1} + \dots + x_{net_k} \leq \sigma(G(i, j) \sim G(i, j + 1)) \quad \forall net_k \text{ that use } G(i, j) \sim G(i, j + 1),$$

$$0 \leq i < W, 0 \leq j < H - 1$$

$$\sum_{net \in Netlist} x_{net_1} + \dots + x_{net_k} \leq \sigma(G(i, j) \sim G(i + 1, j)) \quad \forall net_k \text{ that use } G(i, j) \sim G(i + 1, j),$$

$$0 \leq i < W - 1, 0 \leq j < H$$

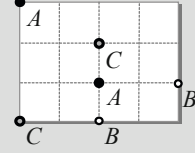
In practice, most pin-to-pin connections are routed using L -shapes or *straight* wires (connections without bends). In this formulation, straight connections can be routed using a straight path or a U -shape; non-straight connections can use both L -shapes. For unrouted nets, other topologies can be found using *maze routing* (Sec. 5.6.3).

ILP-based global routers include *Sidewinder* [5.12] and *BoxRouter 1.0* [5.4]. Both decompose multi-pin nets into two-pin nets using *FLUTE* [5.5], and the route of each net is selected from two alternatives or left unselected. If neither of the two routes available for a net is chosen, *Sidewinder* performs maze routing to find an alternate route and replaces one of the unused routes in the ILP formulation. On the other hand, nets that were successfully routed and do not interfere with unrouted nets can be removed from the ILP formulation. Thus, *Sidewinder* solves multiple ILPs until no further improvement is observed. In contrast, *BoxRouter 1.0* post-processes the results of its ILP using maze-routing techniques.

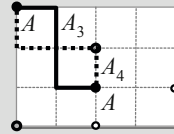
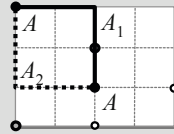
Example: Global Routing Using Integer Linear Programming

Given: (1) nets A - C , (2) an $W = 5 \times H = 4$ routing grid G , (3) $\sigma(e) = 1$ for all $e \in G$, and (4) L -shapes have weight 1.00 and Z -shapes have weight 0.99. The lower-left corner is $(0,0)$.

Task: write the ILP to route the nets in the graph to the right.

**Solution:**

For net A , the possible routes are two L -shapes (A_1, A_2) and two Z -shapes (A_3, A_4).



Net Constraints:

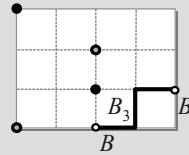
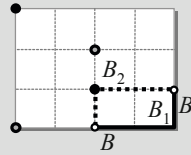
$$x_{A_1} + x_{A_2} + x_{A_3} + x_{A_4} \leq 1$$

Variable Constraints:

$$0 \leq x_{A_1} \leq 1, 0 \leq x_{A_2} \leq 1,$$

$$0 \leq x_{A_3} \leq 1, 0 \leq x_{A_4} \leq 1$$

For net B , the possible routes are two L -shapes (B_1, B_2) and one Z -shape (B_3).



Net Constraints:

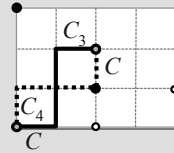
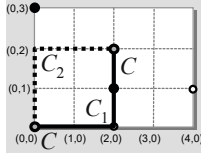
$$x_{B_1} + x_{B_2} + x_{B_3} \leq 1$$

Variable Constraints:

$$0 \leq x_{B_1} \leq 1, 0 \leq x_{B_2} \leq 1,$$

$$0 \leq x_{B_3} \leq 1$$

For net C , the possible routes are two L -shapes (C_1, C_2) and two Z -shapes (C_3, C_4).



Net Constraints:

$$x_{C_1} + x_{C_2} + x_{C_3} + x_{C_4} \leq 1$$

Variable Constraints:

$$0 \leq x_{C_1} \leq 1, 0 \leq x_{C_2} \leq 1,$$

$$0 \leq x_{C_3} \leq 1, 0 \leq x_{C_4} \leq 1$$

Each edge must satisfy capacity constraints. Only non-trivial constraints are shown.

Horizontal Edge Capacity Constraints:

$$G(0,0) \sim G(1,0) : x_{C_1} + x_{C_3} \leq \sigma(G(0,0) \sim G(1,0)) = 1$$

$$G(1,0) \sim G(2,0) : x_{C_1} \leq \sigma(G(1,0) \sim G(2,0)) = 1$$

$$G(2,0) \sim G(3,0) : x_{B_1} + x_{B_3} \leq \sigma(G(2,0) \sim G(3,0)) = 1$$

$$G(3,0) \sim G(4,0) : x_{B_1} \leq \sigma(G(3,0) \sim G(4,0)) = 1$$

$$G(0,1) \sim G(1,1) : x_{A_2} + x_{C_4} \leq \sigma(G(0,1) \sim G(1,1)) = 1$$

$$G(1,1) \sim G(2,1) : x_{A_2} + x_{A_3} + x_{C_4} \leq \sigma(G(1,1) \sim G(2,1)) = 1$$

$$G(2,1) \sim G(3,1) : x_{B_2} \leq \sigma(G(2,1) \sim G(3,1)) = 1$$

$$G(3,1) \sim G(4,1) : x_{B_2} + x_{B_3} \leq \sigma(G(3,1) \sim G(4,1)) = 1$$

$$G(0,2) \sim G(1,2) : x_{A_4} + x_{C_2} \leq \sigma(G(0,2) \sim G(1,2)) = 1$$

$$G(1,2) \sim G(2,2) : x_{A_4} + x_{C_2} + x_{C_3} \leq \sigma(G(1,2) \sim G(2,2)) = 1$$

$$G(0,3) \sim G(1,3) : x_{A_1} + x_{A_3} \leq \sigma(G(0,3) \sim G(1,3)) = 1$$

$$G(1,3) \sim G(2,3) : x_{A_1} \leq \sigma(G(1,3) \sim G(2,3)) = 1$$

Vertical Edge Capacity Constraints:

$$\begin{array}{llll}
G(0,0) \sim G(0,1) & : & x_{C_2} + x_{C_4} & \leq & \sigma(G(0,0) \sim G(0,1)) = 1 \\
G(1,0) \sim G(1,1) & : & x_{C_3} & \leq & \sigma(G(1,0) \sim G(1,1)) = 1 \\
G(2,0) \sim G(2,1) & : & x_{B_2} + x_{C_1} & \leq & \sigma(G(2,0) \sim G(2,1)) = 1 \\
G(3,0) \sim G(3,1) & : & x_{B_3} & \leq & \sigma(G(3,0) \sim G(3,1)) = 1 \\
G(4,0) \sim G(4,1) & : & x_{B_1} & \leq & \sigma(G(4,0) \sim G(4,1)) = 1 \\
G(0,1) \sim G(0,2) & : & x_{A_2} + x_{C_2} & \leq & \sigma(G(0,1) \sim G(0,2)) = 1 \\
G(1,1) \sim G(1,2) & : & x_{A_3} + x_{C_3} & \leq & \sigma(G(1,1) \sim G(1,2)) = 1 \\
G(2,1) \sim G(2,2) & : & x_{A_1} + x_{A_4} + x_{C_1} + x_{C_4} & \leq & \sigma(G(2,1) \sim G(2,2)) = 1 \\
G(0,2) \sim G(0,3) & : & x_{A_2} + x_{A_4} & \leq & \sigma(G(0,2) \sim G(0,3)) = 1 \\
G(1,2) \sim G(1,3) & : & x_{A_3} & \leq & \sigma(G(1,2) \sim G(1,3)) = 1 \\
G(2,2) \sim G(2,3) & : & x_{A_1} & \leq & \sigma(G(2,2) \sim G(2,3)) = 1
\end{array}$$

Objective Function:

$$\begin{array}{ll}
\text{Maximize} & x_{A_1} + x_{A_2} + 0.99 \cdot x_{A_3} + 0.99 \cdot x_{A_4} \\
& + x_{B_1} + x_{B_2} + 0.99 \cdot x_{B_3} \\
& + x_{C_1} + x_{C_2} + 0.99 \cdot x_{C_3} + 0.99 \cdot x_{C_4}
\end{array}$$

► 5.7.2 Rip-Up and Reroute (RRR)

Modern ILP solvers help advanced ILP-based global routers to successfully complete hundreds of thousands of routes within hours [5.4][5.12]. However, commercial EDA tools require greater scalability and lower runtimes. These performance requirements are typically satisfied using the *rip-up and reroute (RRR)* framework, which focuses on problematic nets. If a net cannot be routed, this is often due to physical obstacles or other routed nets being in the way. The key idea is to allow *temporary* violations, so that all nets are routed, but then iteratively remove some nets (rip-up), and route them differently (reroute) so as to decrease the number of violations. In contrast, *push-and-shove* strategies [5.16] move currently routed nets to new locations (without rip-up) to relieve wire congestion or to allow previously unroutable nets to become routable.

An intuitive, greedy approach to routing would route nets sequentially and insist on violation-free routes where such routes are possible, even at the cost of large detours. On the other hand, the RRR framework allows nets to (temporarily) route through over-capacity regions.⁵ This helps decide which nets should detour, rather than detouring the net routed most recently. In the example of Fig. 5.20(a), assume that the nets are routed in an order based on the size of the net's aspect ratio and MBB ($A-B-C-D$). If each net is routed without violations (Fig. 5.20(b)), then net D is forced to detour heavily. However, if nets are allowed to route with violations, then some nets are ripped up and rerouted, enabling D to use fewer routing segments (Fig. 5.20(c)).

⁵ Allowing temporary violations is a common tactic for routing large-scale modern (ASIC) designs, while routing nets without violations is common for PCBs.